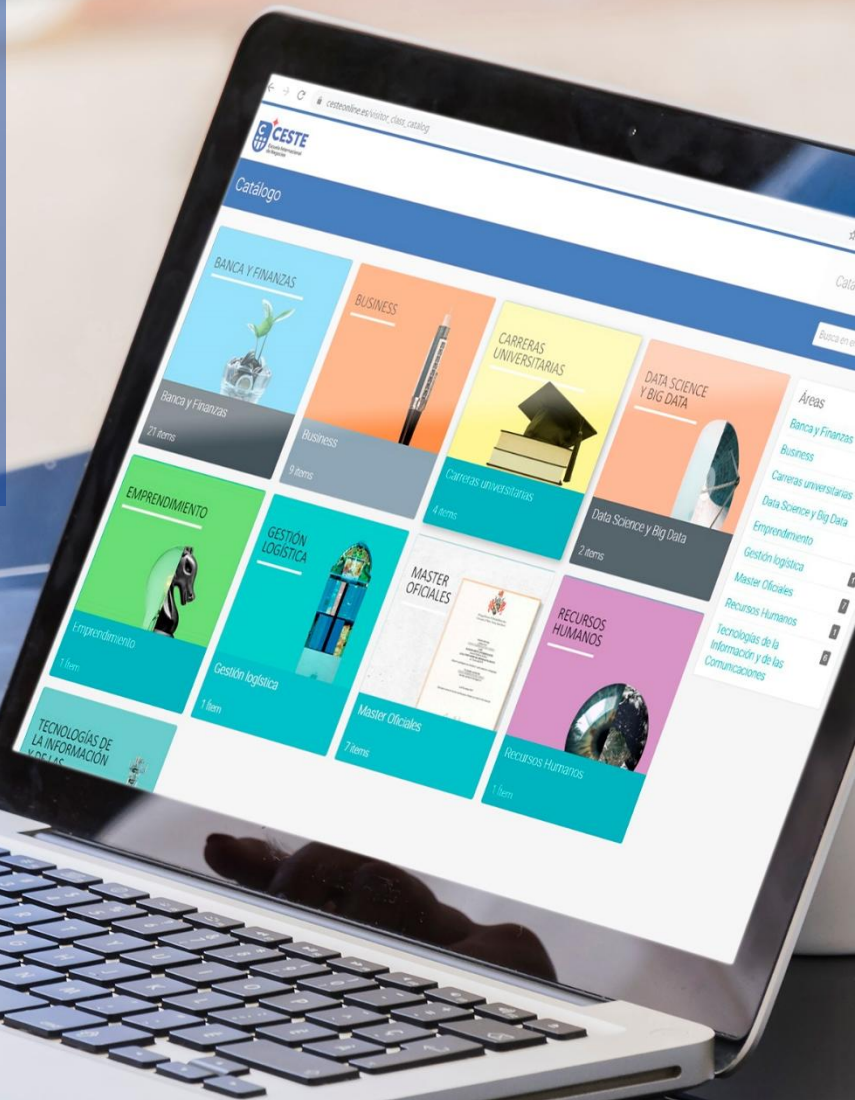


CESTE

Escuela Internacional de Negocios

Zaragoza (España)



Introducción a Curso | Delta Lake avanzado

1. **Bloque I: Fundamentos profundos y arquitectura interna Delta Lake**
 1. Hitos de Databricks
 2. Data Lake to Data Lakehouse
 3. Delta Lake – Partes Principales
 4. Delta Lake – Advance Features
2. **Bloque II: Delta Lake en el modelo Lakehouse: Control Plane y Data Plane**
 1. Delta Lake – Optimizacion Layout
 2. Data Plan vs Control Plan
 3. External Tables vs Managed Tables
 4. Delta Tables Set up
3. **Bloque III: Delta Lake como motor unificado de Ingesta Incremental y Streaming**



Programa de hoy

1. Data Lakehouse

1. Hitos de Databricks
2. Data Lake to Data Lakehouse

2. Delta Lake – Partes Principales

1. Qué es
2. Framework
3. Delta Log
4. Ejemplos
5. DEMO I

3. Delta Lake - Advance Features

1. Data skipping
2. Time Travel
3. Optimize & Z-Order
4. Vacuum
5. DEMO II

4. Kahoot

DATA LAKEHOUSE

1. Hitos de Databricks
 1. Delta Lake 2018
 2. Databricks 2015 (previous Delta Lake)
2. The promise of the Data Lake
 1. Challenge 1
 2. Challenge 2
 3. Challenge 3
 4. Challenge 4
 5. Separated Worlds
3. Data Lakehouse Architecture

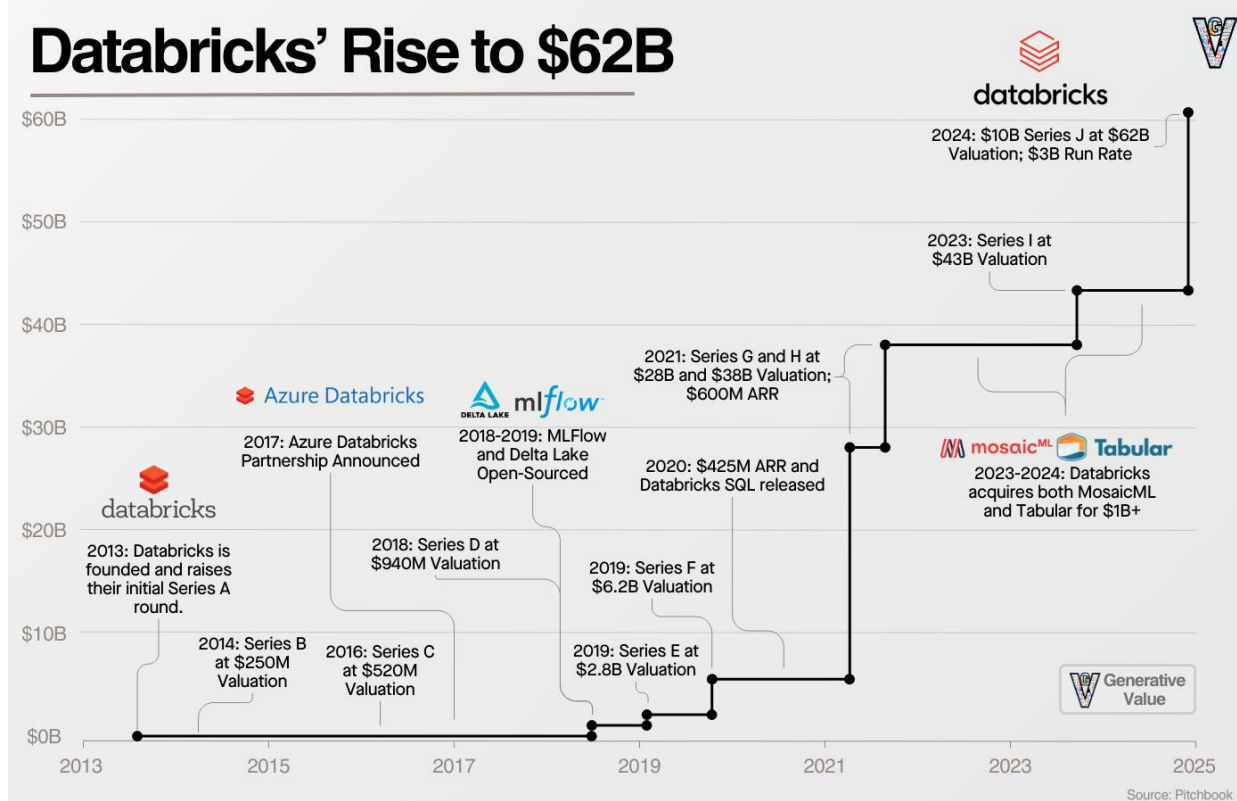
Hitos Databricks | Nacimiento de SPARK

Spark a bit of History...



Technical aspects...

- Spark trabaja en memoria.
- Motor distribuido de procesamiento que permite trabajar en paralelo sobre grandes volúmenes de datos.
- Soporta cualquier tipo de lenguaje Scala, Python, R, SQL & Java
- Permite procesamiento tanto en Batch como en Streaming
- Puede trabajar con datos estructurados, semi estructurados y no estructurados



2020

2010

1980



Histos Databricks | Databricks 2015 “easiest way to use big data”

CAPA 1 — ALMACENAMIENTO

Object Storage

Azure Data Lake Storage · Amazon S3 · Google Cloud Storage



Azure



AWS S3



GCS

CAPA 2 — FORMATO

Apache Parquet

Formato columnar open-source, compresión eficiente, schema embebido



CAPA 3 — PROCESAMIENTO

Apache Spark

Motor distribuido para batch, streaming y ML — sucesor de MapReduce



CAPA 4 — CATÁLOGO

Hive Metastore

Registra tablas virtuales apuntando a rutas del object storage



2015

antes de Delta Lake

"We believe Databricks is the easiest way to use big data."

— Databricks, 2015

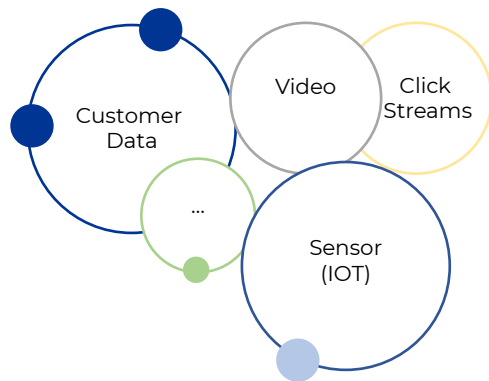
LIMITACIÓN CLAVE

Sin capa transaccional.
El Hive Metastore apunta a un directorio — Spark lee todos los ficheros que encuentra.

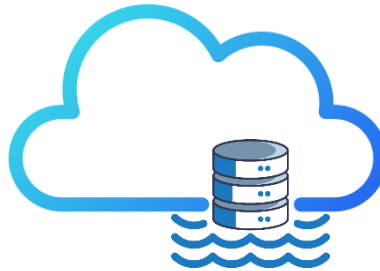
Sin ACID. Sin rollback.
Sin time travel.

The Promise of the Data Lake | Store Data > Think > Process

1 Collect Everything



2 Store it all in the Data Lake



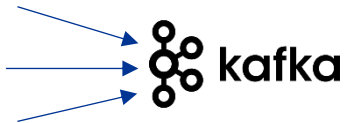
3 Data Science & Machine Learning



- Recommendation Engines
- Risk & Fraud Detection
- IoT & Predictive Maintenance
- Genomics & DNA Sequencing

En un Data Warehouse el schema debe definirse antes de ingestar cualquier dato. El Data Lake invierte esa lógica: almacenas primero, estructuras después. Esto es clave porque el valor real de un dato no siempre es evidente en el momento en que se genera — y cuando lo descubres, el dato ya está ahí.

The Promise of the Data Lake | What does a Data Lake Project look like?



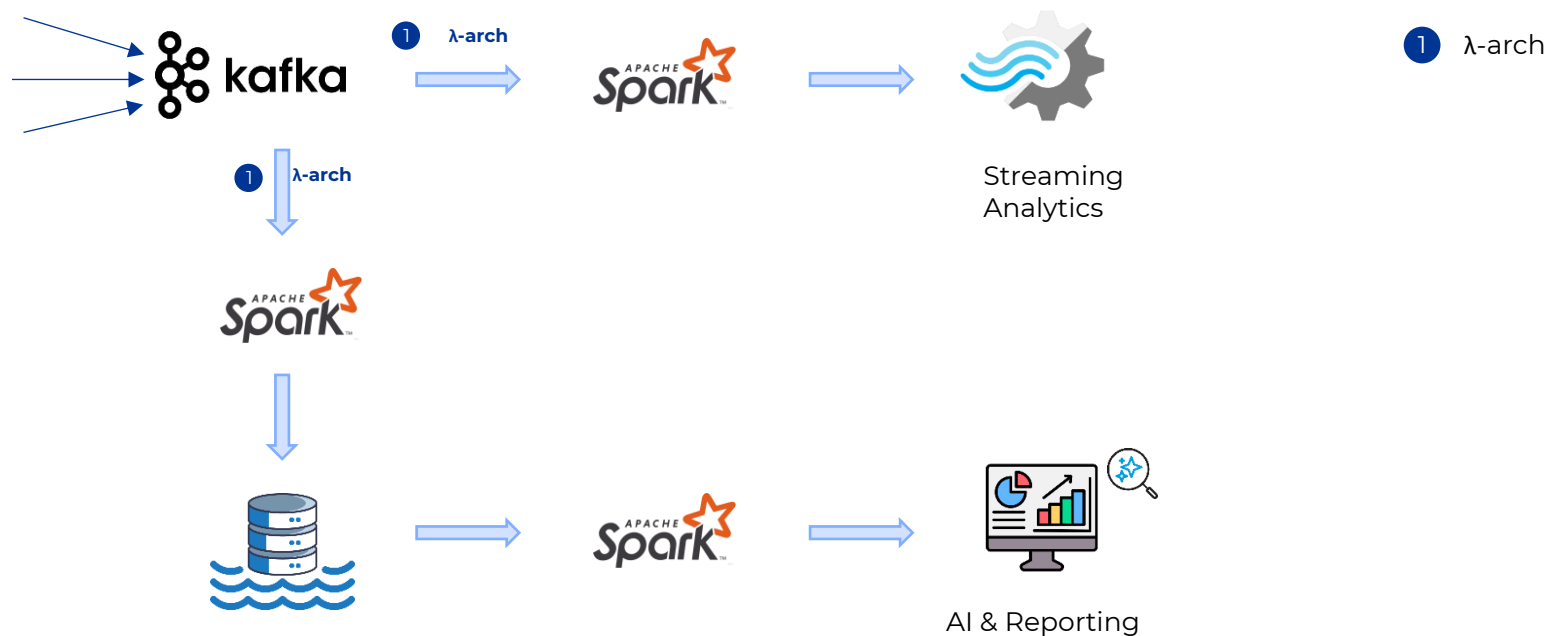
Streaming
Analytics



AI & Reporting

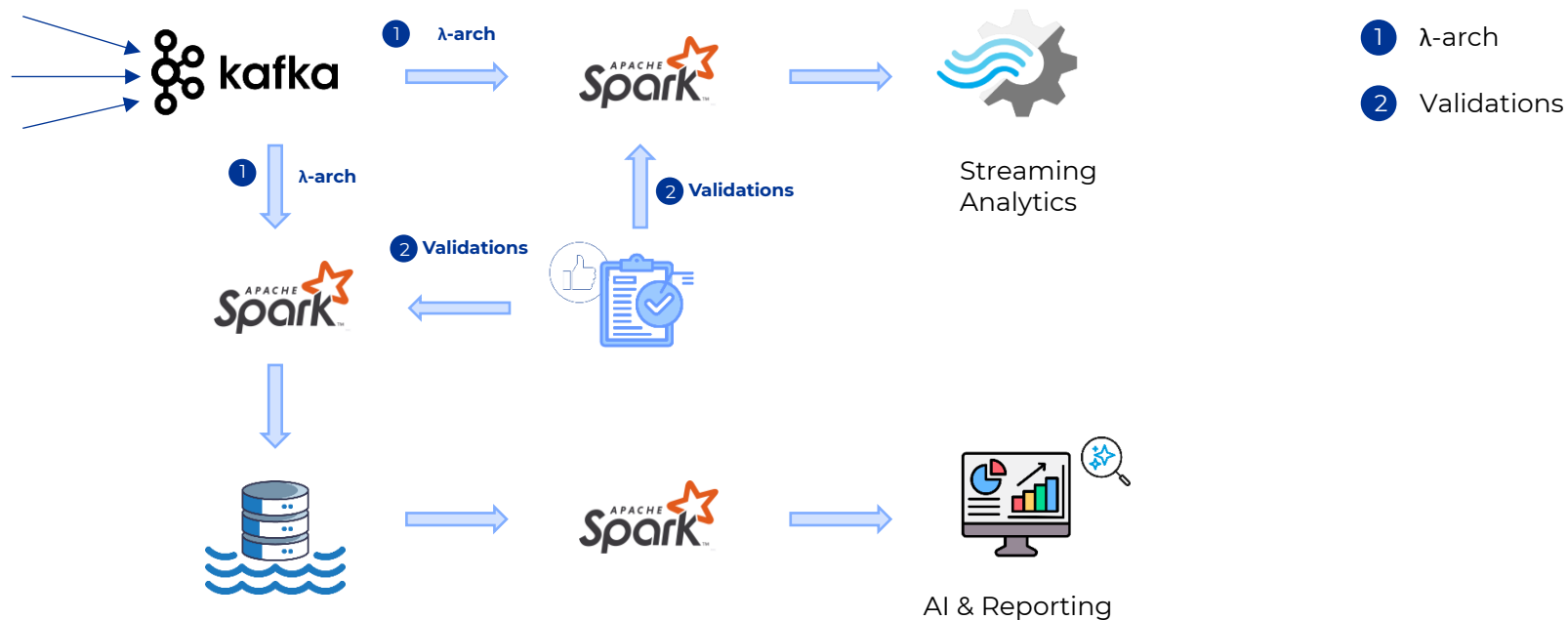
Un escenario habitual en la empresa es recibir un flujo continuo y creciente de eventos desde distintas fuentes, y necesitar alimentar con él dos objetivos a la vez: el análisis en streaming para decisiones en tiempo real, y el uso de los datos históricos acumulados para reporting e inteligencia artificial.

The Promise of the Data Lake | Challenge 1: Streaming + Historical Data



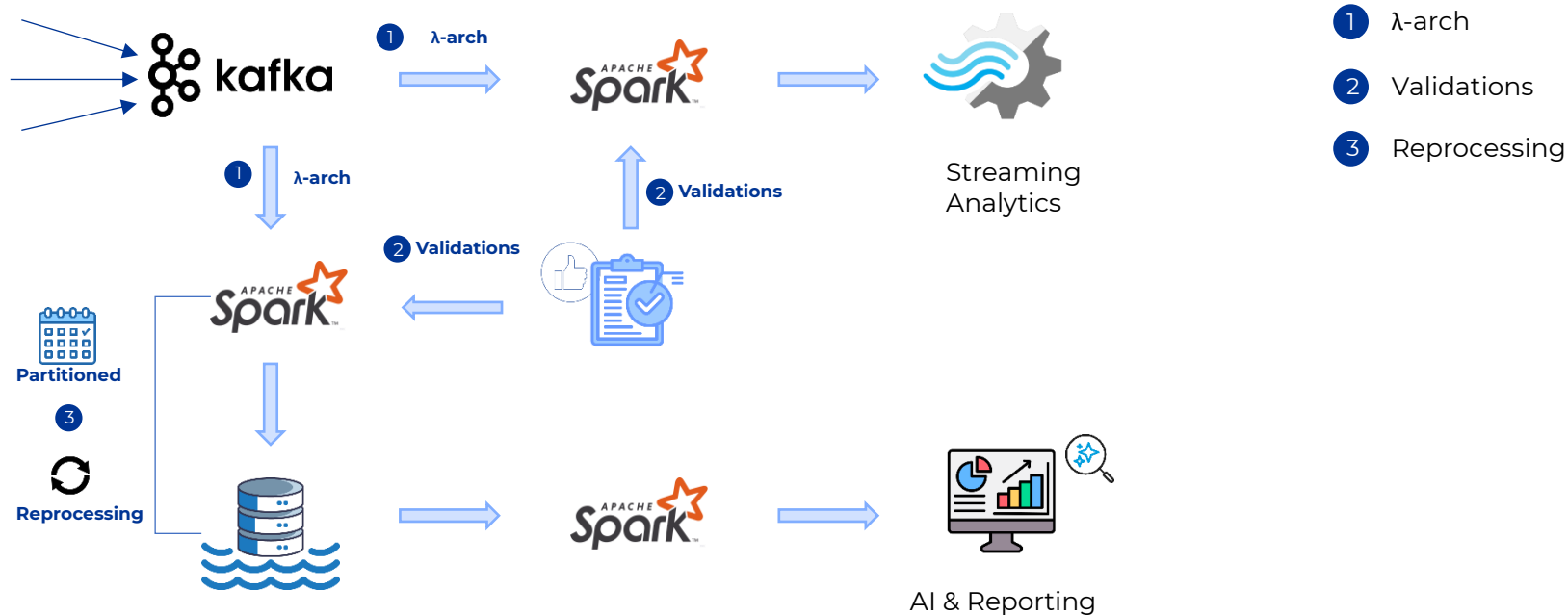
La arquitectura **Lambda** resuelve la necesidad de tener **streaming y reporting** histórico a la vez: una batch layer que reprocesa todos los datos históricos, una speed layer que cubre los datos más recientes en tiempo real, y una serving layer que combina ambas. El coste es que la misma lógica debe implementarse dos veces, en batch y en streaming.

The Promise of the Data Lake | Challenge 2: Messy Data



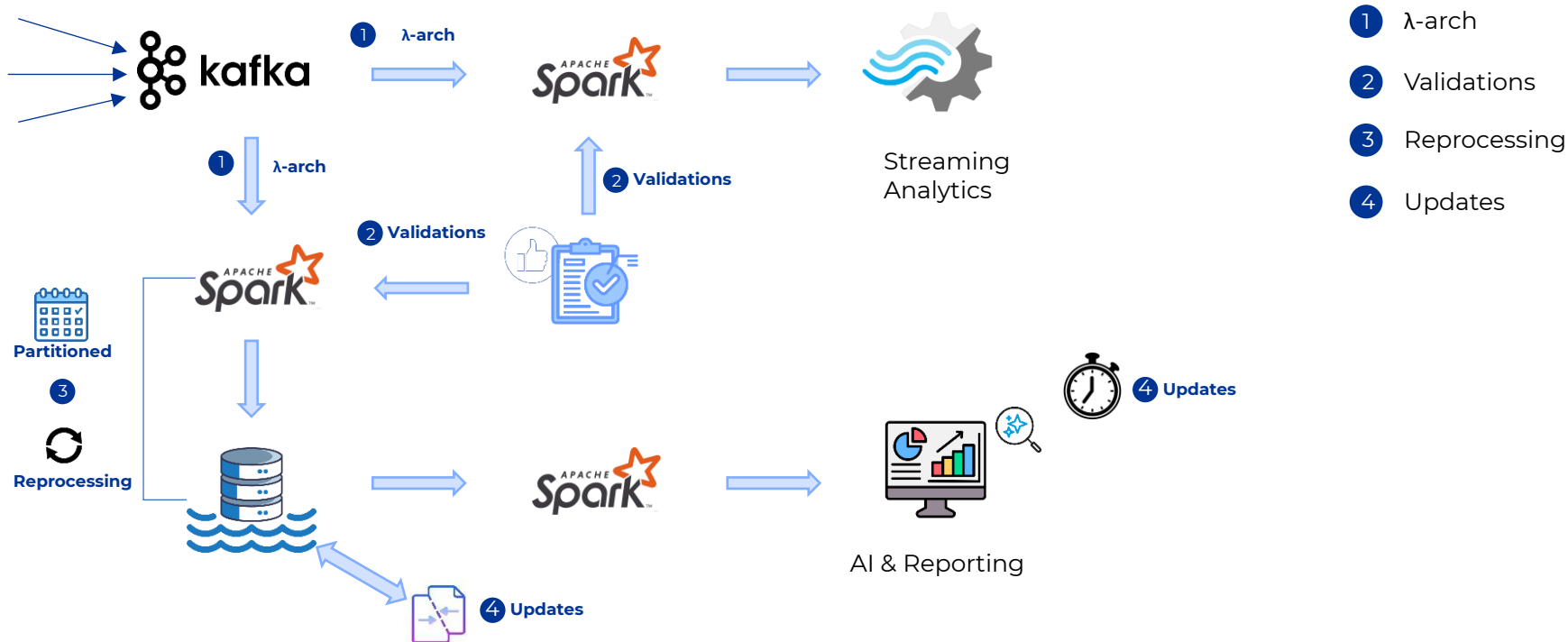
Los datos recibidos raramente llegan limpios, por lo que es **necesario implementar validaciones** que detecten y gestionen problemas antes de que se propaguen a las capas downstream.

The Promise of the Data Lake | Challenge 3: Mistakes & Failures



Las validaciones ayudan, pero siempre existe el riesgo de que un job falle a mitad de ejecución y deje resultados corruptos en el Data Lake. Un patrón habitual para mitigarlo es **trabajar por particiones** en lugar de sobre la tabla completa: si una partición queda en mal estado, se elimina ese directorio y se reprocessa desde cero, sin afectar al resto de los datos.

The Promise of the Data Lake | Challenge 4: Updates



Añadir datos es sencillo, pero actualizar datos existentes es difícil: por GDPR, errores, anonimización. La solución correcta requiere un job de Spark complejo. Lo que se hace en la práctica es reescribir la tabla entera, que funciona pero es costoso. Y si alguien lee la tabla mientras se reescribe, ve resultados inconsistentes — así que se programa de madrugada. Funciona, pero es un parche.

The Promise of the Data Lake | Challenge 3 - 2º approach

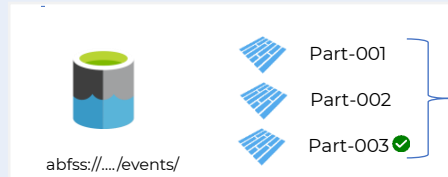
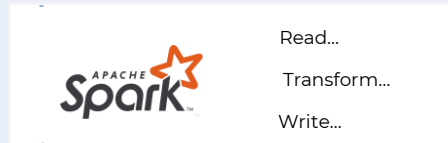
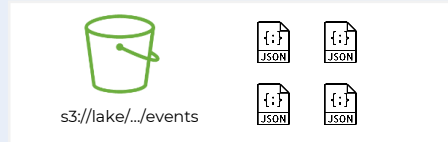


table: events
location: abfss://.../events/
schema: (id,user,timestamp)

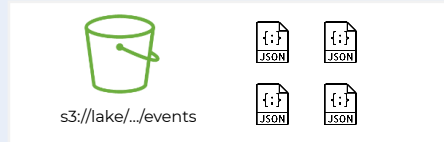


Part-001
Part-002

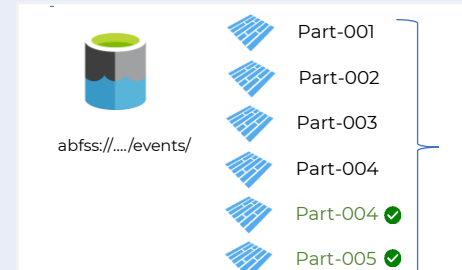
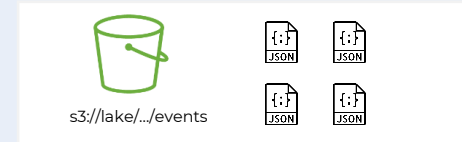
ETL_1 - Interacion 1º ✓



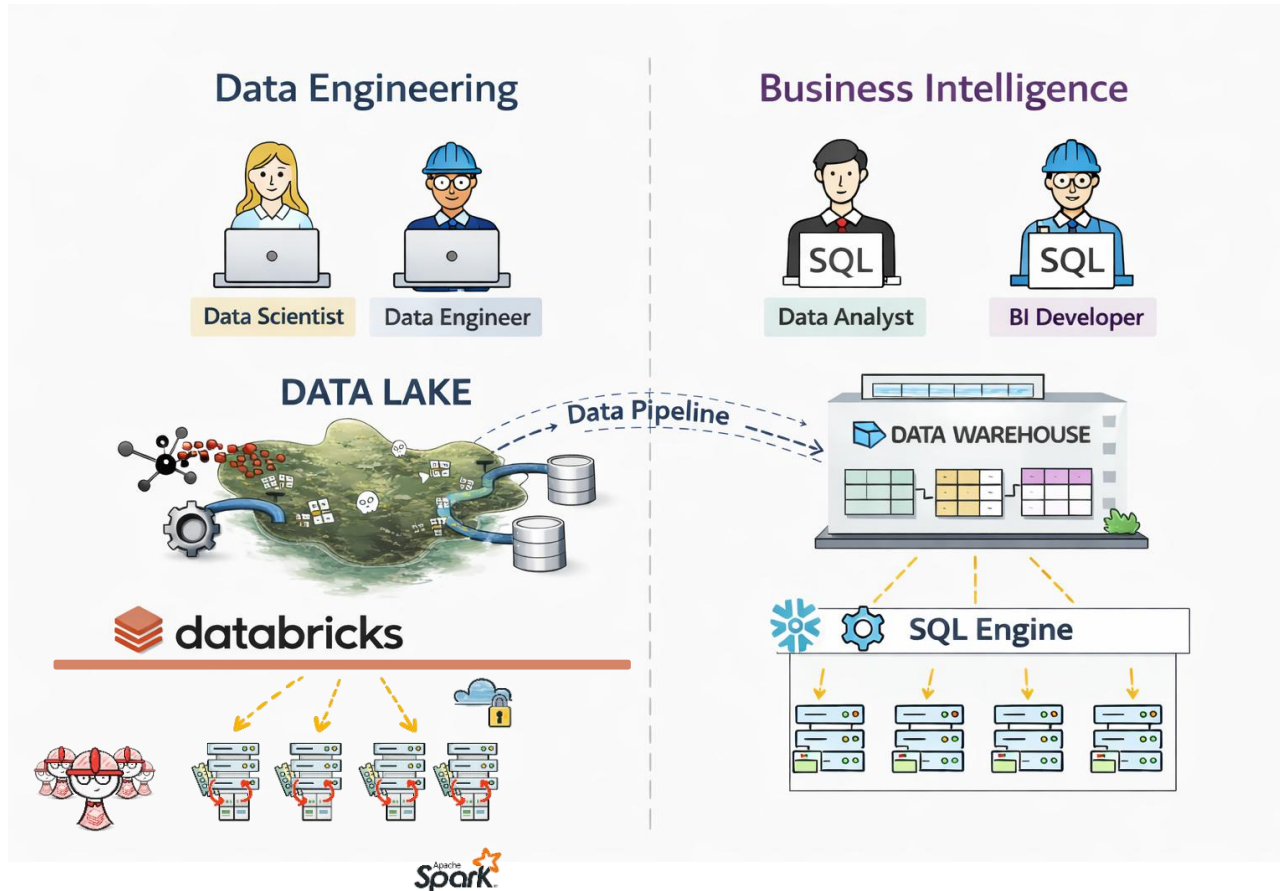
ETL_2 - Iteracion 1º ✗



ETL_2 - Iteracion 2º ✓



The Promise of the Data Lake | separated worlds



The Promise of the Data Lake | Conclusions pre Delta Lake

01 — TRANSACCIONES

ACID sobre object storage

Parquet + S3/ADLS no tienen transacciones. Si un job falla a mitad, los datos quedan en estado corrupto sin rollback. Delta usa el transaction log (`_delta_log`) para garantizar atomicidad.

`_delta_log`

02 — MODIFICACIONES

MERGE, UPDATE y DELETE reales

Sin Delta, borrar una fila implica reescribir particiones enteras sin garantía de atomicidad. Con Delta tienes DML completo — esencial para GDPR y pipelines con correcciones de histórico.

`MERGE / DELETE`

03 — RECUPERACIÓN

Time travel y auditoría

Cualquier error en producción se recupera volviendo a una versión anterior. También permite auditar qué datos existían en un momento exacto — imposible en un lake clásico.

`VERSION AS OF`

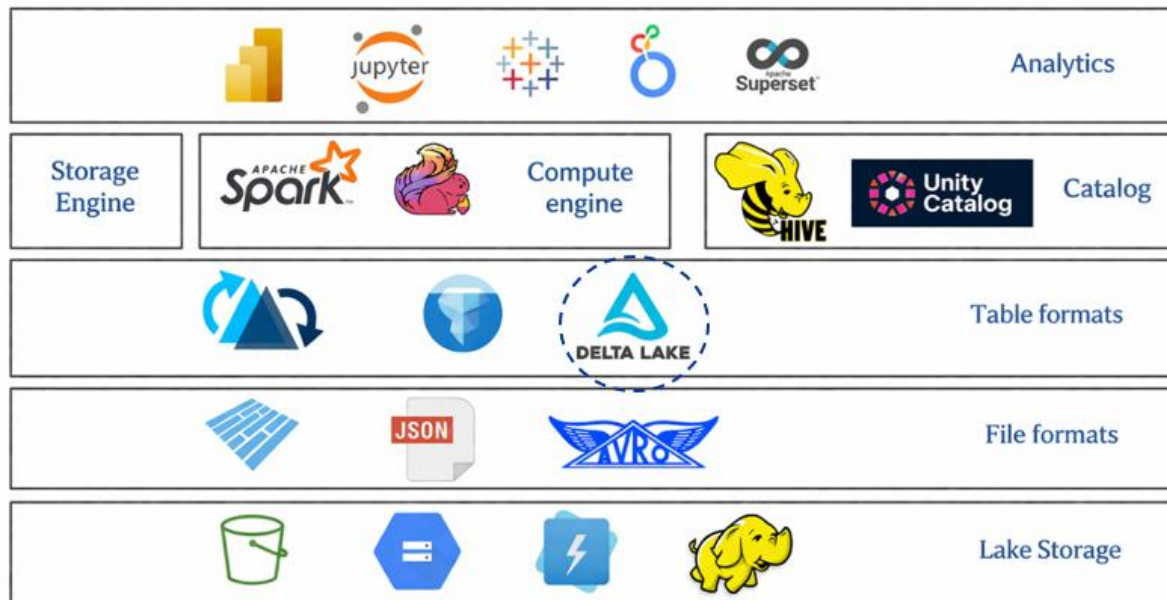
04 — CONTRATOS DE DATOS

Schema enforcement y evolution

El lake clásico acepta cualquier Parquet. Delta rechaza escrituras que violen el schema registrado, protegiendo a los consumidores downstream. La evolución del schema es explícita.

`mergeSchema`

Data Lakehouse | Architecture



01 ACID sobre object storage

El `_delta_log` convierte ficheros Parquet en tablas transaccionales. Lecturas consistentes para BI sin mover datos al DWH.

02 Time travel y recuperación

Cada versión del dato queda registrada. Rollback ante errores, auditoría temporal y reproducibilidad de pipelines.

03 Schema enforcement

Rechaza escrituras que violen el contrato de datos. Protege a todos los consumidores downstream — BI, ML, streaming.

04 Batch + Streaming unificados

Una misma tabla Delta recibe escrituras de Spark batch, Kafka y Flink. Elimina la necesidad de pipelines duplicados.

Data Lakehouse | Key words



01 El logro Real de Databricks

El mérito de Databricks no es solo haber acuñado el término Data Lakehouse, sino haberlo hecho técnicamente viable. **El logro clave es convertir un Data Lake —archivos almacenados en disco— en algo con las garantías propias de un Data Warehouse.** Eso se consigue con una idea que, en su momento, resultó radical: el disco barato puede ser el sistema de registro principal si le añadimos una capa de inteligencia.

02 Qué hace que algo sea Lakehouse

Un Data Lakehouse existe cuando, sobre archivos en object storage, tienes transacciones ACID, control de esquema, metadatos consistentes, lecturas eficientes para BI, concurrencia entre múltiples lectores y escritores, y separación entre storage y compute. Ninguna de esas propiedades la da el disco por sí solo. Las aporta una capa de tabla inteligente que se sitúa encima del almacenamiento.

03 ¿Es Delta Lake imprescindible?

No. Pero fue el primero en demostrar el concepto a escala, y sigue siendo la implementación de referencia. Existen tres tecnologías que hacen posible el Lakehouse: **Delta Lake, Apache Iceberg y Apache Hudi.** Delta Lake fue la primera en llevar transacciones ACID serias a los data lakes, funcionar bien tanto para cargas de BI como de ML, y ser adoptada de forma masiva en entornos productivos. Está totalmente integrada con Apache Spark y es la que convirtió el concepto en realidad industrial. Apache Iceberg nace en Netflix y destaca especialmente en control y evolución de esquemas y en interoperabilidad entre motores. Es igual de válida para construir un Lakehouse moderno, y hoy está muy presente en arquitecturas abiertas. Apache Hudi está orientada principalmente a ingesta incremental y procesamiento near real-time. Es más compleja de operar y tiene menos foco en BI puro que las dos anteriores.

04 Como lo hacen, la pieza técnica clave

Tomando Delta Lake como ejemplo, la clave está en lo que introduce sobre los archivos en disco: transacciones ACID, esquemas controlados, versionado de datos con time travel, consistencia garantizada, optimización de lecturas y metadatos fiables. Todo eso sin modificar el disco ni mover los datos a ninguna base de datos propietaria. El almacenamiento subyacente —object storage estándar— permanece intacto. **La inteligencia no está en dónde se guardan los datos, sino en la capa que los gestiona.**

ARTÍCULOS DE INTERÉS

Evolución de Databricks en el tiempo

Databricks and the Future of Data

An in-depth look at Databricks' vision and its role shaping the future of data platforms.

https://www.generativevalue.com/p/databricks-and-the-future-of-data-369?utm_source=chatgpt.com

Databricks (Pre IPO) Primer and a Requiem for Hadoop

Analysis of Databricks' growth and how it displaced Hadoop as the de facto big data standard.

<https://newsletter.tidalwaveresearch.com/p/databricks-pre-ipo-primer-and-a-requiem>

WHAT IS Databricks?

A clear, accessible explainer of what Databricks is and why it matters for data engineers.

https://dtyped.com/what-is-databricks/?utm_source=chatgpt.com

Databricks — Company Deep Dive

Contrary Research's comprehensive profile: business model, competition, and market position.

https://research.contrary.com/company/databricks?utm_source=chatgpt.com

databricks.com · Javier Martinez

DELTA LAKE

1. Delta Lake - Qué es Delta Lake
2. Delta Lake - Framework
3. Delta Lake - Transaction Log
4. Delta Lake - Ejemplos

Delta Lake | Que es y que no es



Open-source storage framework (marco de almacenamiento de código abierto) que aporta fiabilidad a los lagos de datos.

Permite

time travel

upserts

merges optimizados

data quality checks

ideal para pipelines medallion.

Delta Lake es / no es...

Es...



Tecnología Open Source (Apache License 2.0)



Capa / Framework de almacenamiento sobre object storage



Permite la construcción de un Lakehouse

No es...



Tecnología privada (es open source, no de Databricks)



Formato o medio de almacenamiento (usa Parquet)

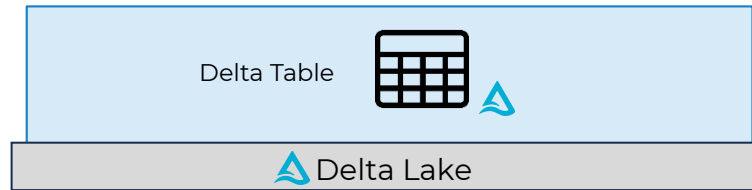


Servicio de Data Warehouse o Data Lake

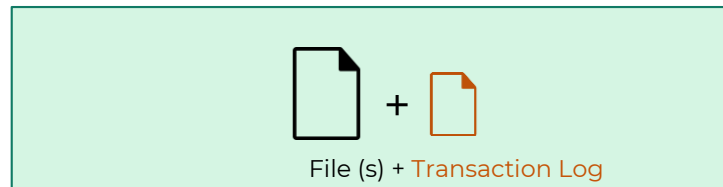
Delta Lake es la capa de inteligencia que convierte un Data Lake en un Lakehouse.

Delta Lake | Framework

Cluster



Cloud Storage



- Delta Lake es un componente que se despliega en el cluster como parte del tiempo de ejecución de Databricks.
- Al crear una tabla delta, esta se almacena en uno o varios ficheros de datos en formato parquet, pero junto con estos archivos, Delta almacena también un registro de transacciones.

Transaction Log

(Delta Log)



Fuente
única de
verdad

El registro de transacciones de Delta Lake es la pieza central que hace posibles las garantías ACID sobre archivos en object storage:

01 Registro ordenado de operaciones

Cada transacción realizada sobre la tabla queda anotada en orden, desde su creación.

02 Fuente única de verdad

Cada vez que Spark accede a la tabla, consulta el Delta Log para conocer su versión más reciente.

03 Archivos JSON por transacción

Cada commit genera un archivo JSON con la operación (INSERT, UPDATE, DELETE) y los predicados aplicados.

04 Ficheros afectados

El log incluye exactamente qué ficheros Parquet se añadieron o eliminaron en cada operación.

El Delta Log es el corazón de Delta Lake: sin el, no hay ACID, no hay time travel, no hay schema enforcement.

Delta Lake | Ejemplo I: Write & Reads



Writer
Process



Reader
Process

Write Data

Add transaction log

Read
Transaction Log

Read
Data Files 1&2

Data Files



File1.parquet



File2.parquet

_delta_log



000.json

Delta Lake | Ejemplo II: Update



Writer
Process



Reader
Process

Update

Add transaction log

Read
Transaction Log

Read
Data Files 2&3

Copy & update



Data Files

File1.parquet File2.parquet File3.parquet

_delta_log



000.json



001.json

001.json knows that file1 is no longer part of the table, so it has to use file2 and file3 instead.

Delta Lake | Ejemplo III: Simultaneous Write & Reads



Writer
Process



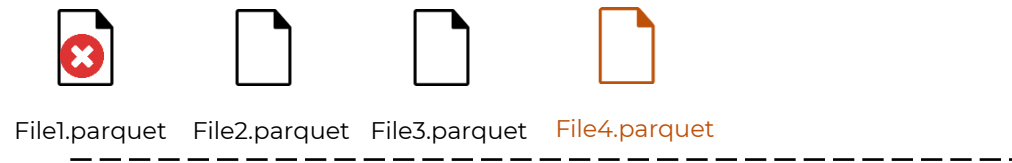
Reader
Process

Write data file4

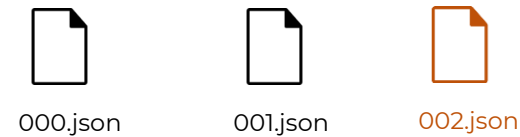
Add Transaction Log



Data Files



_delta_log



Delta Lake | Ejemplo IV: Failed Reads



Writer
Process

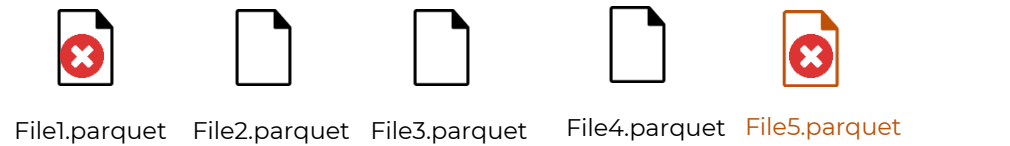


Reader
Process

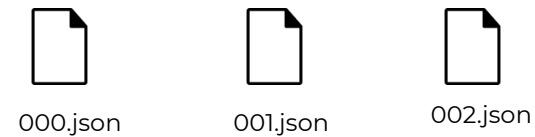
Write data file5



Data Files



_delta_log



DEMO 1.2

UNDERSTANDING DELTA TABLE

DEMO 1.2.B
DIAGRAMA ETL (MIRO)

DELTA LAKE

Advance Features

1. Data Skipping
2. Time Travel
3. Optimize & Z-Order
4. Vacuum

DELTA LAKE

Características avanzadas

01

Data Skipping

Lectura selectiva de ficheros usando estadísticas min/max del transaction log

02

Time Travel

Acceso a versiones históricas de la tabla mediante el Delta Log

03

OPTIMIZE: Compacting Small Files & Z-Ordering

Compactación de ficheros pequeños y reordenación física para maximizar data skipping

04

VACUUM

Limpieza de ficheros obsoletos no referenciados en el transaction log

Delta Lake | Advance Features

Data Skipping

leer menos, responder igual

Al escribir datos, Delta Lake guarda automaticamente en el transaction log las estadísticas min/max de cada archivo. Antes de leer, Spark consulta esas estadísticas y descarta los archivos que matematicamente no pueden contener los datos buscados.



Data Warehouse tradicional

Requiere crear y mantener índices explícitamente. El ingeniero decide qué columnas indexar y cuándo.

Delta Lake

Las estadísticas se recopilan automáticamente en cada write. Sin configuración. Data skipping ocurre de forma transparente en cada query. (32 col por defecto en DLog)

Data skipping es el mecanismo. Z-Ordering es la optimización que hace que ese mecanismo sea realmente efectivo.

Delta Lake | Advance Features

Time Travel

Delta Lake versiona automaticamente cada operacion realizada sobre una tabla, proporcionando un historico completo de cambios.

01 Consultar el historico

Ver todas las versiones y operaciones registradas sobre la tabla:

```
1 DESCRIBE HISTORY table_name
```

Devuelve: version, timestamp, operacion, usuario, duracion y parametros de cada commit.



historico de versiones

02 Leer una version especifica

Accede al estado exacto de la tabla en cualquier version o timestamp:

```
1 SELECT * FROM table_name  
2 VERSION AS OF 5
```

Tambien por timestamp:

```
1 SELECT * FROM table_name  
2 TIMESTAMP AS OF '2024-01-01'
```

Util para auditorias, debug y comparar resultados entre versiones.

03 Rollback / Recuperacion

Restaura la tabla a una version anterior de forma completa y atomica:

```
1 RESTORE TABLE table_name  
2 TO VERSION AS OF 5
```

VACUUM y Time Travel

Tras ejecutar VACUUM no es posible viajar a versiones anteriores al retention period. Los ficheros ya no existen.

El rollback es una operacion ACID: si algo falla durante la restauracion, la tabla queda intacta.

Time travel funciona gracias al Delta Log. VACUUM lo limita. Conocer ambos es clave para usarlos bien.

OPTIMIZE

Delta Lake puede mejorar significativamente el rendimiento de lectura optimizando como se almacenan los datos en disco. El comando OPTIMIZE compacta pequenos ficheros en otros mas grandes, reduciendo el numero de operaciones I/O.

Comando

```
1 OPTIMIZE table_name
```



Menos ficheros

Reduce el numero de ficheros a escanear en cada query



Lecturas rapidas

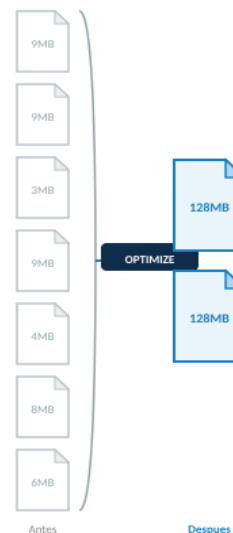
Menos I/O = consultas mas rapidas en BI y ML



Rendimiento general

Mejor performance global de la tabla Delta

Que sucede con los ficheros



Los ficheros originales quedan como obsoletos en el log hasta que VACUUM los elimine.

OPTIMIZE no cambia los datos, cambia como estan distribuidos en disco.

Z-Order

Z-ORDER reorganiza los datos colocando valores similares en los mismos ficheros. No crea subdirectorios (a diferencia del partitioning). Se aplica como extension del comando OPTIMIZE.

Comando

```
1 OPTIMIZE table_name
2 ZORDER BY (fecha, planta_id)
```

Conexion con Data Skipping

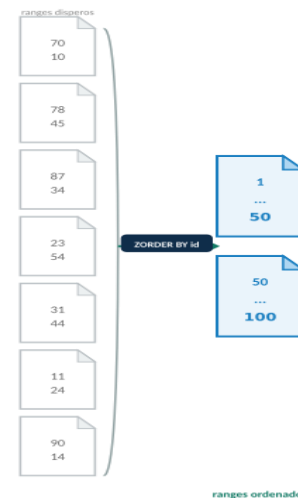
Si consultas id = 30, Delta Lee las estadísticas del log:

- 1 El valor esta en el fichero 1 (min=1, max=50)
- 2 No necesita leer el fichero 2 (min=50, max=100)

Resultado

Mucho menos I/O. Consultas considerablemente mas rapidas.

Efecto sobre los ficheros (por columna id)



Cuando usar Z-Order

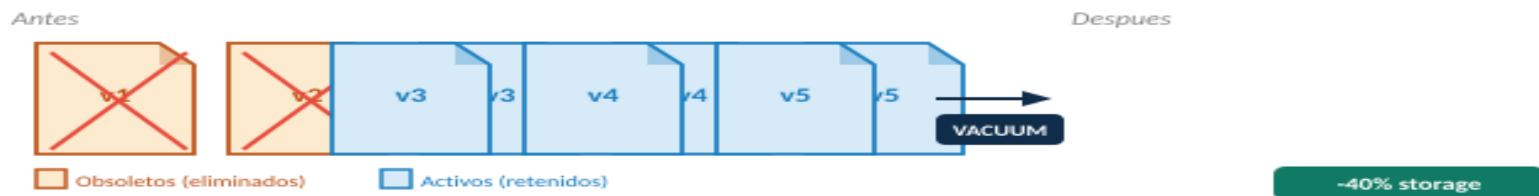
- Columnas usadas frecuentemente en filtros (WHERE)
- Columnas numericas o de alta cardinalidad
- Tablas grandes con muchas consultas de lectura

Data skipping es el mecanismo. Z-Order es lo que hace que ese mecanismo sea realmente efectivo.

VACUUM

gestion del almacenamiento en Delta Lake

Delta Lake mantiene multiples versiones de los datos para garantizar ACID y time travel. Esto genera ficheros obsoletos que ocupan espacio sin ser referenciados en el transaction log.



Uso basico

```
VACUUM table_name
```

Por defecto conserva los ultimos 7 dias de ficheros. Protege operaciones largas que aun puedan estar leyendo versiones anteriores.

Retention Period personalizado

```
VACUUM table_name RETAIN 168 HOURS
```

Elimina todos los ficheros anteriores al umbral indicado. 168h = 7 dias. Puedes reducirlo con cuidado.

! Impacto en Time Travel

Tras ejecutar VACUUM no se puede hacer time travel a versiones anteriores al retention period. Los ficheros ya no existen fisicamente.

Buenas practicas

No reducir el retention period sin conocer el impacto. Evitar VACUUM agresivo en produccion. Coordinar con pipelines y procesos batch largos.

VACUUM libera espacio, pero elimina la posibilidad de time travel. Usalo con criterio.

DEMO 1.3
DELTA TABLE ADVANCE FEATURES